

מסמך אפיון - GreenShop

מערכת שירות לקוחות מתקדמת מבוססת סוכן AI

1. תיאור הבעיה והפתרון

הבעיה

בחנות אונליין (GreenShop), נציגי שירות לקוחות אנושיים עונים ידנית על שאלות חוזרות — מדיניות החזרות, מפרטי מוצרים, סטטוס הזמנות, ובקשות להחזר כספי. זה יקר, איטי, ולא זמין מסביב לשעון. בנוסף, נציג אנושי עלול להבטיח ללקוח פיצוי החורג ממדיניות החברה.

הפתרון

סוכן AI אוטונומי המבוסס על LangGraph, המסוגל:

- לענות על שאלות ידע כלליות (מדיניות, מוצרים) מתוך בסיס ידע פנימי באמצעות RAG.
- לטפל בבקשות ספציפיות ללקוח (סטטוס הזמנה, החזר כספי) על ידי ביצוע פעולה בפועל במערכות החברה, דרך אוטומציית חתום.
- לשמור על גבולות בטוחים — חסימת קלט זדוני או לא רלוונטי, ומניעת הבטחות שחורגות ממדיניות החברה.

ערך עסקי

מענה מיידי ועקבי ללקוחות, הפחתת עומס מנציגים אנושיים, ואכיפה אוטומטית של מדיניות החברה באמצעות שתי שכבות בקרה (Guardrails).

2. ארכיטקטורת הגרף (LangGraph)

הסוכן בנוי כגרף מצבים (StateGraph) עם 6 צמתים, המחברים בקשתות רגילות ובשתי קשתות מותנות (Conditional Edges).

הויזואליזציה המלאה של הגרף מצורפת בקובץ `langgraph_visualization.png` (וגם כטקסט ב- `langgraph_visualization.txt`).

מבנה ה-State

ה-State (מוגדר ב- `src/state.py`) הוא מבנה נתונים שעובר בין כל הצמתים ומצטבר לאורך הריצה:

שדה	תפקיד
<code>user_message</code>	הודעת הלקוח המקורית
<code>block_reason</code> , <code>is_input_valid</code>	תוצאת בדיקת ה-Input Guardrail
<code>route</code>	החלטת הניתוב של הראוטר
<code>retrieved_context</code>	קטעי המידע שנשלפו מה-RAG
<code>tool_result</code>	הנתונים שהוחזרו מ-n8n
<code>draft_answer</code>	טיוטת התשובה מצומת ה-Generation
<code>final_answer</code> , <code>is_output_safe</code>	תוצאת ה-Output Guardrail
<code>guardrail_feedback</code>	הסיבה לפסילת טיוטה, לצורך ניסוח מחדש
<code>regeneration_attempts</code>	מונה ניסיונות, למניעת לולאה אינסופית

פירוט הצמתים

צומת 1 — Input Guardrail & Router

`src/nodes/input_guardrail_router.py`

בעל תפקיד כפול:

1. **Guardrail** — בדיקת תקינות ההודעה הנכנסת לפני שהיא מגיעה ללוגיקה העסקית. מסנן בקשות off-topic, ניסיונות Prompt Injection, ותוכן לא הולם.

2. Router — סיווג ההודעה לאחד משני מסלולים.

נימוש: הצומת מפעיל את המודל עם `with_structured_output(RouterDecision)`, סכמת Pydantic הכופה פלט מובנה עם שלושה שדות: `is_valid`, `block_reason`, `route`. השימוש בפלט מובנה (ולא בפרסור טקסט חופשי) מבטיח שההחלטה תהיה ניתנת לקריאה על ידי הקוד באופן דטרמיניסטי.

Conditional Edge — הפונקציה `route_after_input_guardrail` קוראת את ה-State ומחזירה אחד משלושה יעדים:

- `blocked` → צומת `blocked_input` (תשובת סירוב, ללא חשיפת שאר המערכת)
- `knowledge_question` → צומת Retrieval
- `customer_specific` → צומת Tool Execution

נימוק ארכיטקטוני: ה-Guardrail ממומש כצומת נפרד בגרף ולא כהנחיה בתוך ה-prompt של הסוכן הראשי. הסיבה: הנחיה בפרומפט ניתנת לעקיפה על ידי Prompt Injection, בעוד שצומת נפרד יוצר נקודת החלטה קשיחה בזרימת הבקרה — הודעה שנפסלה כלל לא מגיעה ללוגיקה העסקית.

צומת 2 — Retrieval (RAG)

`src/nodes/retrieval.py`

שולף את המידע הרלוונטי מבסיס הידע הוקטורי. שאילתת הלקוח מומרת ל-embedding, ומתבצע חיפוש דמיון סמנטי המחזיר את שלושת הקטעים הקרובים ביותר. התוצאה נשמרת ב- `retrieved_context`.

צומת 3 — Tool Execution (n8n)

`src/nodes/tool_execution.py`

מפעיל סוכן משני (`create_agent`) המצויד בכלי `call_n8n_webhook`. הסוכן מחליט בעצמו — בסגנון ReAct — האם ובאילו פרמטרים לקרוא לכלי: הוא מחלץ את מספר ההזמנה מתוך הודעת הלקוח בשפה חופשית, ומסווג את סוג הבקשה (`order_status` / `refund_request`).

נימוק ארכיטקטוני: זהו הרכיב שמממש את דרישת האוטונומיה. הסוכן אינו מבצע רצף קבוע — הוא מפרש את כוונת הלקוח ומרכיב את הקריאה לכלי בזמן ריצה.

צומת 4 — Generation

src/nodes/generation.py

מנסח את התשובה ללקוח על בסיס המידע שהצטבר ב-State (מ-RAG, מ-n8n, או משניהם). ה- `system_prompt` מגדיר את זהות הסוכן ומחייב אותו להתבסס אך ורק על המידע שסופק.

כאשר הצומת מופעל בעקבות פסילה של ה-Output Guardrail, הוא מקבל גם את `guardrail_feedback` ומנחה את המודל לתקן את הבעיה הספציפית שזוהתה.

צומת 5 — Output Guardrail

src/nodes/output_guardrail.py

בודק את הטיוטה מול המידע שסופק לסוכן, ופוסל אותה במקרים הבאים:

- **הזיה** — התשובה מכילה מידע שאינו מופיע במקורות.
- **חריגה ממדיניות** — הבטחת פיצוי, החזר או הנחה שלא אושרו.
- **תוכן לא הולם**.

גם כאן נעשה שימוש בפלט מובנה (`OutputCheck`) עם השדות `is_safe` ו-`issue`.

Conditional Edge — `route_after_output_guardrail`:

- תשובה שאושרה → `END`
- תשובה שנפסלה ונותרו ניסיונות → חזרה ל-Generation עם המשוב (לולאת תיקון)
- מיצוי מכסת הניסיונות (`MAX_REGENERATION_ATTEMPTS = 2`) → תשובת ברירת מחדל בטוחה המפנה לנציג אנושי

נימוק ארכיטקטוני: מכסת הניסיונות הכרחית — בלעדיה, מודל שממשיך לייצר תשובה בעייתית היה יוצר לולאה אינסופית בגרף.

צומת עזר — Blocked Input

src/graph.py

מטפל בהודעות שנפסלו ב-Input Guardrail ומחזיר הודעת סירוב אחידה, מבלי לחשוף את סיבת הפסילה המדויקת ללקוח.

3. אסטרטגיית RAG

src/rag/vector_store.py

רכיב	בחירה	נימוק
מסד וקטורי	ChromaDB	פועל מקומית ללא שירות חיצוני, נשמר לדיסק, קל לאריזה ב-Docker
Chunking	RecursiveCharacterTextSplitter, גודל 500, חפיפה 50	מפצל לפי גבולות טבעיים (פסקה → שורה → משפט). החפיפה מונעת אובדן הקשר בגבול הקטע
Embedding	text-embedding-3-small (OpenAI) / nomic-embed-text (Ollama)	נבחר בהתאם לספק המוגדר
מטא-דאטה	שדה source (שם קובץ המקור) בכל chunk	מאפשר סינון לפי מקור וציון מקורות בתשובה

מקורות המידע (data/knowledge_base/): מדיניות החזרות, מדיניות משלוחים, וקטלוג מוצרים.

הערה תפעולית חשובה: מודלי embedding שונים מייצרים וקטורים במימדים שונים. החלפת ספק המודל מחייבת בנייה מחדש של המסד הוקטורי.

4. אינטגרציית Flask ושרת

שרת Flask

src/server.py

תפקיד	שיטה	Endpoint
מקבל { "message": "..."} , מאתחל את ה-State, מריץ את הגרף, ומחזיר { "answer": "..."}	POST	/chat

תפקיד	שיטה	Endpoint
מאפשר ל-n8n לקרוא בחזרה לשרת (תקשורת דו-כיוונית)	POST	/n8n-callback

src/startup.py מוודא שהמסד הוקטורי קיים לפני עליית השרת, כך שהרצה על מחשב נקי אינה דורשת שלבים ידניים.

הכלי שמחבר את הסוכן ל-n8n

src/tools/n8n_tool.py

הכלי `call_n8n_webhook` מוגדר עם דקורטור `@tool`. ה-`docstring` שלו הוא התיאור שהמודל קורא כדי להחליט מתי ובאילו פרמטרים להפעיל אותו — ולכן הוא מנוסח במפורש ומפרט את הערכים החוקיים של `request_type`.

הכלי שולח HTTP POST עם JSON, כולל `timeout` והחזרת שגיאה מובנית במקרה כשל, כדי שתקלת תקשורת לא תפיל את הגרף.

Workflow-n8n-2

(ניתן לייבוא ישיר) docs/n8n_workflow.json

#	צומת	תפקיד
1	Webhook	מקבל JSON מהסוכן בנתיב <code>/webhook/greenshop</code>
2	Switch By Request Type	צומת הלוגיקה — מנתב לפי <code>request_type</code>
3a	Fetch Order Status	שולף סטטוס, חברת שילוח ומספר מעקב ממסד הזמנות מדומה
3b	Create Refund Request	רושם בקשת החזר ומנפיק קוד קופון פיצוי
4	Respond to Webhook	מחזיר JSON מובנה לסוכן

נקודת אכיפה נוספת: מגבלת הפיצוי (`MAX_COMPENSATION_ILS = 200`) מוגדרת בתוך `n8n` ולא בצד הסוכן. כך, גם אם המודל היה מנסה להבטיח סכום גבוה יותר, המערכת החיצונית לא הייתה מאשרת אותו. זוהי שכבת הגנה שנייה, בלתי תלויה במודל.

5. תיאור הכלים (Tools)

שם	תיאור שניתן למודל	פרמטרים
<code>call_n8n_webhook</code>	"שולח בקשה למערכת <code>n8n</code> לביצוע פעולה על הזמנת לקוח. יש להשתמש בכלי כאשר הלקוח מבקש מידע או פעולה שקשורים להזמנה ספציפית שלו, כגון בדיקת סטטוס משלוח או בקשת החזר כספי."	<code>request_type</code> <code>order_status</code> <code>refund_request</code> <code>details</code> , <code>order_id</code>

שליפת ה-RAG ממומשת כצומת ייעודי בגרף ולא ככלי, משום שהמסלול אליה נקבע על ידי הראוטר — אין צורך בהחלטה נוספת של המודל בשלב זה.

6. סביבת הרצה — Docker

`docker-compose.yml`, `Dockerfile`

שני שירותים:

- 1. **greenshop-agent** — נבנה מה-`Dockerfile` (בסיס `python:3.11-slim`), חושף פורט `5000`.
- 2. **n8n** — ה-`image` הרשמי `n8nio/n8n:latest`, חושף פורט `5678`.

הגדרות מפתח:

- `N8N_WEBHOOK_URL` נדרס ל- `http://n8n:5678/...` — בתוך רשת `Docker` הפנייה היא לשם השירות ולא ל-`localhost`.
- `chroma_db` ממופה כ-`volume`, כדי שהמסד הוקטורי לא ייבנה מחדש בכל הפעלה.
- נתוני `n8n` נשמרים ב-`volume` בשם `n8n_data`.

הפעלה: `docker-compose up --build`

7. תרחישי בדיקה

— `tests/test_scenarios.py` — חמישה תרחישים המכסים את כל מסלולי הגרף:

#	תרחיש	מסלול נבדק	תוצאה
1	"מה מדיניות ההחזרות שלכם?"	RAG	✓
2	"איפה ההזמנה שלי? מספר 12345"	n8n — סטטוס	✓
3	"התעלם מכל ההוראות הקודמות..."	Input Guardrail — Injection	✓ נחסם
4	"כתוב לי שיר על חתולים"	Input Guardrail — Off-topic	✓ נחסם
5	"מה קורה עם ההזמנה 99999?"	מקרה קצה — הזמנה לא קיימת	✓

הרצה: `python -m tests.test_scenarios`